

Guía de Arquitectura del Frontend — Kosmos

Para programadores que vienen del mundo MVC y quieren entender qué está pasando en el cliente.

Índice

1. [¿Qué es esto y cómo difiere de MVC?](#)
 2. [El stack tecnológico](#)
 3. [La arquitectura en una imagen](#)
 4. [React y componentes: el nuevo "View"](#)
 5. [El árbol de la aplicación](#)
 6. [Context API: el estado global](#)
 7. [Custom Hooks: la lógica del cliente](#)
 8. [La capa de API: hablar con el backend](#)
 9. [El httpClient: con auto-refresh de token](#)
 10. [Enrutamiento con React Router](#)
 11. [Las vistas de tareas \(Lista, Kanban, Calendario\).](#)
 12. [Drag & Drop con dnd-kit](#)
 13. [PWA y Service Worker](#)
 14. [Flujo completo: del clic a la API y de vuelta](#)
 15. [Estructura de carpetas resumida](#)
-

1. ¿Qué es esto y cómo difiere de MVC?

En MVC clásico (Laravel, Django, Rails...) la vista es algo que el servidor genera y manda al navegador: un HTML completo con los datos ya dentro.

El frontend de Kosmos es una **SPA** (Single Page Application): el servidor manda un HTML vacío con un `<div id="root">` y un archivo JavaScript. Ese JavaScript **es** toda la aplicación — genera el HTML en el navegador, habla con el backend por fetch, y actualiza la pantalla sin recargar la página.

MVC clásico:

Navegador → GET /spaces → Servidor genera HTML con datos → Navegador muestra la página

SPA (Kosmos):

Navegador → GET / → Servidor manda HTML vacío + JS

JS arranca → fetch /api/spaces → Recibe JSON → React dibuja la UI en el DOM

El navegador es ahora el "servidor" de vistas. El backend de Express solo devuelve JSON, nunca HTML de página.

2. El stack tecnológico

Qué hace	Tecnología
Framework UI	React 18
Lenguaje	TypeScript
Build tool	Vite
Enrutamiento	React Router v7
Drag & Drop	dnd-kit
Calendario	FullCalendar
PWA	vite-plugin-pwa + Workbox
Estilos	CSS custom properties (sin framework CSS)

3. La arquitectura en una imagen



4. React y componentes: el nuevo "View"

En MVC hay un archivo de plantilla por vista (`.blade.php` , `.html.erb` , etc.). En React, la UI se construye con **componentes**: funciones TypeScript que devuelven JSX.

JSX es la sintaxis que mezcla HTML y JavaScript:

```
// Esto es TypeScript + JSX (archivo .tsx)
function SpaceCard({ space }: { space: Space }): JSX.Element {
  return (
    <div className="space-card">
      <span>{space.icon}</span>
      <h3>{space.name}</h3>
      <p>{space.isActive ? "Activo" : "Inactivo"}</p>
    </div>
  );
}
```

Las llaves `{ }` ejecutan JavaScript dentro del HTML. Esto reemplaza a las directivas de plantilla (`{ { }` , `@foreach` , etc.) de los motores de template.

Diferencia clave con MVC: los componentes no solo renderizan — también **reaccionan** a cambios de datos. Si `space.name` cambia, React actualiza automáticamente el DOM sin que tú lo pidas.

Estado local con `useState`

La forma de tener datos que cambian dentro de un componente:

```
function SpaceDetailPage(): JSX.Element {
  // priorityFilter es el valor actual; setPriorityFilter es la función para cambiarlo
  const [priorityFilter, setPriorityFilter] = useState<TaskPriority | "ALL">("ALL");
  const [selectedTaskId, setSelectedTaskId] = useState<string | null>(null);

  return (
    <div>
      {/* Al hacer clic, cambia el filtro → React re-renderiza el componente */}
      <button onClick={() => setPriorityFilter("HIGH")}>Solo alta prioridad</button>
    </div>
  );
}
```

Cada vez que se llama `setPriorityFilter( ... )` , React vuelve a llamar la función del componente y actualiza el DOM con los nuevos valores.

5. El árbol de la aplicación

`main.tsx` — El punto de entrada

```
// src/main.tsx
createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
);
```

Equivale al `index.php` o `app.js` de un servidor MVC: es donde todo arranca. `BrowserRouter` activa el sistema de rutas del lado del cliente.

`app.tsx` — El enrutador principal

```
// src/app/app.tsx
export function App(): JSX.Element {
  return (
    <ThemeProvider>      {/* ← estado global del tema */}
    <AuthProvider>       {/* ← estado global del usuario */}
    <Routes>
      <Route path="/login"   element={<LoginPage />} />
      <Route path="/register" element={<RegisterPage />} />

      {/* Rutas protegidas: si no estás autenticado, te mandan a /login */}
      <Route element={<ProtectedRoute />>
        <Route element={<AppLayout />>    {/* ← header + nav en todas las páginas */}
          <Route path="/"          element={<AlmanaquePage />} />
          <Route path="/tablero"    element={<TableroPage />} />
          <Route path="/spaces"     element={<SpacesPage />} />
          <Route path="/spaces/:spaceId" element={<SpaceDetailPage />} />
          <Route path="/tasks"      element={<AllTasksPage />} />
        </Route>
      </Route>

      <Route path="*" element={<Navigate to="/" replace />} />
    </Routes>
  </AuthProvider>
</ThemeProvider>
);
}
```

`ProtectedRoute` — El middleware de autenticación del frontend

```
// src/app/protected-route.tsx
export function ProtectedRoute(): JSX.Element {
  const { authenticated } = useAuth();
  // Si no estás autenticado, redirige a /login
  // Si sí, renderiza la ruta hija (<Outlet />)
  return authenticated ? <Outlet /> : <Navigate to="/login" replace />;
}
```

Equivalente en MVC: el middleware `auth` que redirige a `/login` si no hay sesión. Aquí funciona igual pero en el navegador: no hay petición al servidor.

`AppLayout` — El layout compartido

```
// src/app/app-layout.tsx
export function AppLayout(): JSX.Element {
  const { logout, user } = useAuth();
  const { theme, toggleTheme } = useTheme();

  return (
    <div className="app-shell">
      <header className="app-header">
        {/* Logo + navegación + botón de tema + nombre de usuario + logout */}
        <nav>
          <Link to="/">Almanaque</Link>
          <Link to="/tablero">Tablero</Link>
          <Link to="/spaces">Espacios</Link>
          <Link to="/tasks">Todas las tareas</Link>
        </nav>
      </header>
      <main className="app-main">
        <Outlet /> {/* ← aquí se renderiza la página actual */}
      </main>
    </div>
  );
}
```

`<Outlet />` es el equivalente de `@yield('content')` en Blade o `<%= yield %>` en ERB: el hueco donde se mete la página actual dentro del layout.

6. Context API: el estado global

En MVC el estado de sesión vive en el servidor (session, cookies). En React, el estado global del lado del cliente se gestiona con la **Context API**: un mecanismo para que cualquier componente del árbol acceda a datos sin pasarlos como props manualmente.

AuthContext — Quién está logueado

```
// src/features/auth/auth-context.tsx

// 1. Definir qué datos tendrá el contexto
interface AuthContextValue {
  user: AuthUser | null;
  authenticated: boolean;
  login: (email: string, password: string) => Promise<void>;
  register: (email: string, password: string, name: string) => Promise<void>;
  logout: () => void;
}

// 2. Crear el contexto
const AuthContext = createContext<AuthContextValue | null>(null);

// 3. El Provider envuelve la app y provee los datos
export function AuthProvider({ children }: { children: ReactNode }): JSX.Element {
  const [user, setUser] = useState<AuthUser | null>(() => getStoredUser());
  const [authenticated, setAuthenticated] = useState(isAuthenticated());

  const value = useMemo(() => ({
    user,
    authenticated,
    login: async (email, password) => {
      const result = await authApi.login({ email, password });
      setTokens(result.tokens.accessToken, result.tokens.refreshToken);
      setStoredUser(result.user);
      setUser(result.user);
      setAuthenticated(true);
    },
    logout: () => {
      clearTokens();
      setUser(null);
      setAuthenticated(false);
    },
    // ...
  })), [user, authenticated]);

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

// 4. Hook para consumir el contexto desde cualquier componente
export function useAuth(): AuthContextValue {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used within an AuthProvider");
  return ctx;
}
```

Cómo se usa desde cualquier componente:

```
function AppLayout(): JSX.Element {
  const { user, logout } = useAuth(); // ← sin pasar props, accede directo al contexto
  return <span>{user?.name}</span>;
}
```

```
// src/features/theme/theme-context.tsx
export function ThemeProvider({ children }: { children: ReactNode }): JSX.Element {
  const [theme, setTheme] = useState<Theme>(() => {
    const stored = localStorage.getItem("kosmos-theme");
    return (stored as Theme) ?? "dark";
  });

  useEffect(() => {
    // Cada vez que cambia el tema, actualiza el atributo en el HTML
    document.documentElement.setAttribute("data-theme", theme === "light" ? "light" : "");
    localStorage.setItem("kosmos-theme", theme);
  }, [theme]);

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme: () => setTheme(t => t === "dark" ? "light" : "dark") }}>
      {children}
    </ThemeContext.Provider>
  );
}
```

El tema se persiste en `localStorage` y se aplica añadiendo `data-theme="light"` al `<html>`, lo que activa variables CSS definidas en `global.css`.

7. Custom Hooks: la lógica del cliente

Los **Custom Hooks** son funciones que empiezan por `use` y encapsulan lógica con estado. Son el equivalente de los Servicios de MVC — pero en el cliente.

En MVC harías algo así en el Controller:

```
// En MVC (servidor)
$tasks = Task::where('space_id', $spaceId)→get();
return view('tasks', compact('tasks'));
```

En React, esa lógica vive en un custom hook que el componente llama:


```
// En React (cliente)
export function useTasks(spaceId: string) {
  const [tasks, setTasks] = useState<Task[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  // useEffect = "ejecuta esto cuando el componente monta (o cuando spaceId cambia)"
  useEffect(() => {
    tasksApi.listTasks(spaceId)
      .then(setTasks)
      .catch(err => setError(err.message))
      .finally(() => setLoading(false));
  }, [spaceId]);

  return {
    tasks,
    loading,
    error,
    // También expone las mutaciones:
    createTask: async (input) => {
      const task = await tasksApi.createTask(spaceId, input);
      setTasks(prev => [...prev, task]); // actualiza estado local optimísticamente
    },
    updateStatus: async (taskId, status) => {
      const updated = await tasksApi.updateTaskStatus(spaceId, taskId, status);
      setTasks(prev => prev.map(t => t.id === taskId ? { ...t, ...updated } : t));
    },
    deleteTask: async (taskId) => {
      await tasksApi.deleteTask(spaceId, taskId);
      setTasks(prev => prev.filter(t => t.id !== taskId));
    },
    // ... más mutaciones
  };
}
```

El componente lo usa así, completamente desacoplado de la lógica:

```
function SpaceDetailPage(): JSX.Element {
  const { spaceId } = useParams();
  const { tasks, loading, error, createTask, updateStatus, deleteTask } = useTasks(spaceId!);

  if (loading) return <p>Cargando ... </p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ListView
      tasks={tasks}
      onStatusChange={updateStatus}
      // ...
    />
  );
}
```

Hooks del proyecto

Hook	Archivo	Qué gestiona
<code>useSpaces()</code>	<code>features/spaces/use-spaces.ts</code>	Lista de spaces, crear/editar/borrar/aplicar template
<code>useTasks(spaceId)</code>	<code>features/tasks/use-tasks.ts</code>	Tareas de un space, CRUD, reordenar, subtareas
<code>useMyTasks()</code>	<code>features/tasks/use-my-tasks.ts</code>	Todas las tareas del usuario logueado
<code>useSpaceFilter()</code>	<code>features/spaces/use-space-filter.ts</code>	Filtrado/búsqueda local de spaces
<code>useSpaceTemplates()</code>	<code>features/spaces/use-space-templates.ts</code>	Lista de templates disponibles
<code>usePushNotifications()</code>	<code>features/notifications/use-push-notifications.ts</code>	Suscripción/cancelación de notificaciones push
<code>useAuth()</code>	<code>features/auth/auth-context.tsx</code>	Sesión del usuario (via Context)
<code>useTheme()</code>	<code>features/theme/theme-context.tsx</code>	Tema oscuro/claro (via Context)

Patrón de actualización optimista

En `useTasks` y `useSpaces` las mutaciones siguen este patrón:

1. Usuario hace clic → llamar la mutación del hook
2. Hook llama a la API (fetch al backend)
3. Cuando la API responde con éxito → hook actualiza el estado local con el dato real del servidor
4. React re-renderiza el componente con el nuevo dato

Esto garantiza que la UI siempre refleja lo que hay en el servidor — no hay estado "stale" (desactualizado).

8. La capa de API: hablar con el backend

Cada feature tiene su propio archivo `api.ts` con funciones puras (sin estado) que llaman al backend:

```
// src/features/tasks/api.ts

export function listTasks(spaceId: string): Promise<Task[]> {
  return httpClient.get<Task[]>(`/spaces/${spaceId}/tasks`);
}

export function createTask(spaceId: string, input: { title: string; ... }): Promise<Task> {
  return httpClient.post<Task>(`/spaces/${spaceId}/tasks`, input);
}

export function updateTaskStatus(spaceId: string, taskId: string, status: TaskStatus): Promise<Task> {
  return httpClient.patch<Task>(`/spaces/${spaceId}/tasks/${taskId}/status`, { status });
}

export function deleteTask(spaceId: string, taskId: string): Promise<void> {
  return httpClient.delete<void>(`/spaces/${spaceId}/tasks/${taskId}`);
}
```

Estas funciones son análogas a los métodos de un **Repositorio** o un **Service** en MVC: ocultan el detalle de cómo se hace la petición y exponen una interfaz limpia que los hooks consumen.

9. El httpClient: con auto-refresh de token

El `httpClient` es el objeto central que hace todos los `fetch` al backend. Su característica más interesante es el **refresh automático de JWT**:

```
// src/shared/lib/http-client.ts

// Token de refresco: se guarda un único Promise para evitar múltiples refreshes simultáneos
let refreshPromise: Promise<string | null> | null = null;

async function request<T>(path: string, options: RequestInit = {}, allowRetry = true): Promise<T> {
  // 1. Añade el Bearer token a todos los requests
  const token = getAccessToken();
  const headers = new Headers(options.headers);
  headers.set("Content-Type", "application/json");
  if (token) headers.set("Authorization", `Bearer ${token}`);

  const response = await fetch(`${API_BASE_URL}${path}`, { ...options, headers });

  // 2. Si el servidor responde 401 (token expirado), intenta refrescar automáticamente
  if (response.status === 401 && allowRetry && !PUBLIC_PATHS.has(path)) {
    const newToken = await refreshAccessToken(); // pide un nuevo accessToken
    if (newToken) {
      return request<T>(path, options, false); // reintenta la petición original
    }
    // Si no se pudo refrescar, limpia tokens y redirige al login
    clearTokens();
    window.location.href = "/login";
    throw new HttpError(401, "Sesión expirada");
  }

  if (!response.ok) {
    const body = await response.json().catch(() => ({}));
    throw new HttpError(response.status, body?.error?.message ?? "La solicitud falló");
  }

  return response.json() as T;
}

export const httpClient = {
  get: <T>(path: string) => request<T>(path),
  post: <T>(path: string, body?: unknown) => request<T>(path, { method: "POST", body: JSON.stringify(body) }),
  patch: <T>(path: string, body?: unknown) => request<T>(path, { method: "PATCH", body: JSON.stringify(body) }),
  delete: <T>(path: string) => request<T>(path, { method: "DELETE" }),
};
```

El patrón de **refreshPromise** es importante: si 3 peticiones distintas fallan con 401 al mismo tiempo, el refresh solo se ejecuta una vez (se comparte el mismo Promise). Sin esto harías 3 llamadas de refresh simultáneas con el mismo **refreshToken**.

auth-storage.ts — Persistencia en localStorage

```
// src/shared/lib/auth-storage.ts

const ACCESS_TOKEN_KEY = "kosmos.accessToken";
const REFRESH_TOKEN_KEY = "kosmos.refreshToken";
const USER_KEY = "kosmos.user";

export const setTokens = (access, refresh) => { localStorage.setItem(... ) };
export const getAccessToken = () => localStorage.getItem(ACCESS_TOKEN_KEY);
export const isAuthenticated = () => getAccessToken() !== null;
export const clearTokens = () => { localStorage.removeItem(... ) };
```

Los tokens se guardan en `localStorage` para que persistan entre sesiones (si cierras y abres el navegador, sigues logueado hasta que el token expire o hagas logout).

10. Enrutamiento con React Router

React Router v7 maneja las URLs en el cliente. No hay peticiones al servidor para cambiar de página — JavaScript actualiza la URL y renderiza el componente correspondiente.

```
// Definición de rutas en app.tsx
<Routes>
  <Route path="/login"          element={<LoginPage />} />
  <Route path="/spaces/:spaceId" element={<SpaceDetailPage />} />
</Routes>
```

Para navegar entre páginas:

```
// Componente de link (como <a href> pero sin recargar la página)
<Link to="/spaces">Ir a espacios</Link>

// O programáticamente:
const navigate = useNavigate();
navigate("/spaces");
```

Para leer parámetros de la URL:

```
function SpaceDetailPage(): JSX.Element {
  const { spaceId } = useParams<{ spaceId: string }>();
  // spaceId viene de la URL: /spaces/abc-123 → spaceId = "abc-123"
}
```

11. Las vistas de tareas (Lista, Kanban, Calendario)

`SpaceDetailPage` puede mostrar las tareas en tres vistas diferentes. Todas reciben los mismos datos (`tasks`) y solo cambia la forma de presentarlos:

```
// src/features/spaces/space-detail-page.tsx

// El usuario elige la vista; se guarda en el backend (space.viewType)
{space?.viewType === "LIST" && (
  <ListView tasks={filteredTasks} onStatusChange={updateStatus} onReorder={reorderTasks} />
)}
{space?.viewType === "KANBAN" && (
  <KanbanView tasks={filteredTasks} onStatusChange={updateStatus} />
)}
{space?.viewType === "CALENDAR" && (
  <CalendarView tasks={filteredTasks} onCreateTask={ ... } onUpdateTask={ ... } />
)}
```

Mismo dato, tres representaciones — esto es un patrón muy común en React: el estado vive arriba (en el hook `useTasks`) y los componentes de vista solo lo muestran.

Vista	Descripción	Componente
Lista	Tareas en orden vertical, con drag & drop para reordenar	<code>views/list-view.tsx</code>
Kanban	Columnas por estado (Por hacer / En progreso / Hecho), con drag & drop entre columnas	<code>views/kanban-view.tsx</code>
Calendario	Integración con FullCalendar, tareas como eventos por fecha	<code>views/calendar-view.tsx</code>

12. Drag & Drop con dnd-kit

La vista Lista y la Kanban usan **dnd-kit** para el drag & drop. La librería provee hooks que convierten cualquier elemento del DOM en arrastrable o en zona de drop.

Kanban: mover entre columnas

```
// src/features/tasks/views/kanban-view.tsx

// Hace que una tarjeta sea arrastrable
function KanbanCard({ task, onOpenTask }) {
  const { attributes, listeners, setNodeRef, isDragging } = useDraggable({ id: task.id });

  return (
    <div
      ref={setNodeRef}           // ← registra el DOM node en dnd-kit
      {...listeners}            // ← eventos de mouse/touch para iniciar el drag
      {...attributes}           // ← atributos de accesibilidad (aria-*)
      style={{ opacity: isDragging ? 0.3 : 1 }}
    >
      {task.title}
    </div>
  );
}

// Hace que una columna sea zona de drop
function KanbanColumn({ status, tasks }) {
  const { setNodeRef } = useDroppable({ id: status }); // ← "soy una zona de drop"
  return <div ref={setNodeRef}>{/* tarjetas */}</div>;
}

// El contexto que conecta todo y escucha cuando termina el drag
<DndContext onDragEnd={({event}) => {
  const newStatus = event.over?.id as TaskStatus;
  if (newStatus) onStatusChange(task.id, newStatus);
}}>
  {/* Las columnas y tarjetas van aquí */}
</DndContext>
```

Lista: reordenar con sortable

```
// src/features/tasks/views/list-view.tsx

// SortableContext maneja el orden de los items
<SortableContext items={localIds} strategy={verticalListSortingStrategy}>
  {orderedTasks.map(task => (
    <SortableTaskRow key={task.id} task={task} />
  ))}
</SortableContext>
```

El orden se actualiza **en tiempo real** mientras el usuario arrastra (via `handleDragOver`), y al soltar se manda al backend con `reorderTasks(taskIds)`.

13. PWA y Service Worker

Kosmos es una **Progressive Web App**: se puede instalar en el escritorio o móvil como si fuera una app nativa.

El Service Worker (`sw.ts`)

Un Service Worker es un script que corre en segundo plano en el navegador, independiente de la página:

```
// src/sw.ts

// Workbox precachea todos los assets del build para funcionar offline
precacheAndRoute(self.__WB_MANIFEST);

// Escucha notificaciones push del servidor (via web-push)
self.addEventListener("push", (event) => {
  const payload = event.data?.json() ?? { title: "Kosmos", body: "" };
  event.waitUntil(
    self.registration.showNotification(payload.title, {
      body: payload.body,
      icon: "/icons/icon-192.png",
    }),
  );
});

// Al hacer clic en la notificación, enfoca o abre la app
self.addEventListener("notificationclick", (event) => {
  event.notification.close();
  event.waitUntil(self.clients.matchAll({ type: "window" })).then(clients => {
    if (clients.length) return clients[0].focus();
    return self.clients.openWindow("/");
  });
});
```

Flujo de notificaciones push

1. Usuario activa notificaciones en la app
2. Frontend pide permiso al navegador → obtiene una "PushSubscription" (endpoint + claves)
3. Frontend envía esa suscripción al backend (POST /api/notifications/subscribe)
4. Backend la guarda en la DB (tabla push_subscriptions)
5. Cada minuto, el scheduler del backend busca tareas próximas
6. Backend usa web-push para mandar la notificación al endpoint del navegador
7. Service Worker recibe el evento "push" y muestra la notificación al usuario

14. Flujo completo: del clic a la API y de vuelta

Veamos qué ocurre cuando el usuario hace clic en "Crear tarea" en `SpaceDetailPage` :

1. Usuario escribe "Comprar leche" y hace clic en "Agregar"
2. QuickAddTask llama a: createTask({ title: "Comprar leche" })
 - └─ viene de useTasks(spaceId).createTask
3. useTasks.createTask:
 - └─ llama a tasksApi.createTask(spaceId, { title: "Comprar leche" })
4. tasksApi.createTask:
 - └─ llama a httpClient.post("/spaces/abc/tasks", { title: "Comprar leche" })
5. httpClient.post:
 - └─ fetch("http://localhost:3000/api/spaces/abc/tasks", {
 - method: "POST",
 - headers: { Authorization: "Bearer <token>", "Content-Type": "application/json" },
 - body: '{"title":"Comprar leche"}'
6. Backend recibe la petición:
 - └─ requireAuth verifica el JWT → extrae userId
 - └─ TaskController.create() → valida con Zod → llama a CreateTaskUseCase
 - └─ CreateTaskUseCase: verifica ownership → crea entidad Task → guarda en DB
 - └─ Responde: 201 { id: "xyz", title: "Comprar leche", status: "TODO", ... }
7. httpClient recibe la respuesta → parsea el JSON → devuelve el objeto Task
8. useTasks.createTask recibe la nueva tarea:
 - └─ setTasks(prev ⇒ [...prev, nuevaTarea])
 - └─ React detecta que el estado cambió → re-renderiza SpaceDetailPage
9. ListView renderiza las tareas incluyendo la nueva "Comprar leche"
 - └─ El DOM se actualiza – el usuario ve la tarea aparecer

Todo este flujo sucede en menos de 200ms. No hay recarga de página.

15. Estructura de carpetas resumida

```

src/
  main.tsx          ← punto de entrada: monta React en el DOM
  sw.ts             ← Service Worker (PWA + notificaciones push)
  vite-env.d.ts     ← tipos de las variables de entorno (VITE_*)
  styles/
    global.css      ← variables CSS para temas, estilos base
  app/
    app.tsx         ← rutas + providers globales
    app-layout.tsx  ← header + nav + <Outlet />
    protected-route.tsx ← guarda de autenticación
  shared/
    lib/
      http-client.ts ← fetch wrapper con Bearer token + auto-refresh JWT
      auth-storage.ts ← wrapper de localStorage para tokens y usuario
  features/
    auth/
      api.ts         ← login(), register() → llaman al backend
      auth-context.tsx ← AuthProvider + useAuth()
      login-page.tsx ← página de login
      register-page.tsx ← página de registro
    spaces/
      api.ts         ← listSpaces(), createSpace(), toggleSpace() ...
      types.ts       ← interfaces TypeScript: Space, SpaceViewType
      use-spaces.ts   ← hook: estado + mutaciones de spaces
      use-space-filter.ts ← hook: filtrado/búsqueda local
      use-space-templates.ts ← hook: lista de templates predefinidos
      spaces-page.tsx ← página: listado de todos los spaces
      space-detail-page.tsx ← página: detalle de un space (con las 3 vistas)
      space-card.tsx  ← componente: tarjeta de un space
      space-filter-bar.tsx ← componente: barra de búsqueda
      create-space-form.tsx ← componente: formulario de creación
      apply-template-menu.tsx ← componente: menú de templates
    tasks/
      api.ts         ← listTasks(), createTask(), updateTaskStatus() ...
      types.ts       ← interfaces: Task, Subtask, TaskStatus, TaskPriority
      use-tasks.ts   ← hook: estado + CRUD + reorder + subtareas
      use-my-tasks.ts ← hook: todas las tareas del usuario
      date-utils.ts  ← helpers para formatear fechas
      quick-add-task.tsx ← componente: input rápido para crear tarea
      task-detail-modal.tsx ← componente: modal con detalle, edición, subtareas
      all-tasks-page.tsx ← página: todas las tareas del usuario
      almanaque-page.tsx ← página: vista de agenda/próximas tareas
      tablero-page.tsx ← página: tablero kanban global
    views/
      list-view.tsx  ← vista Lista con drag & drop para reordenar
      kanban-view.tsx ← vista Kanban con drag & drop entre columnas
      calendar-view.tsx ← vista Calendario con FullCalendar
      day-detail-modal.tsx ← modal de detalle de día en el calendario
  theme/
    theme-context.tsx ← ThemeProvider + useTheme()
  notifications/
    api.ts          ← subscribe(), unsubscribe()
    push-utils.ts    ← helpers para registrar el Service Worker
    use-push-notifications.ts ← hook: gestión del estado de suscripción
    notification-toggle.tsx ← componente: botón activar/desactivar notificaciones

```

Resumen de la separación de responsabilidades

Archivo / carpeta	Responsabilidad	Análogo en MVC
<code>features/*/api.ts</code>	Llamadas HTTP al backend	Repository / API Client
<code>features/*/use-*.ts</code>	Estado + mutaciones	Service / Controller
<code>features/*/*.tsx</code>	Renderizado de UI	View / Template
<code>features/*/types.ts</code>	Tipos TypeScript	DTO / Model (solo estructura)
<code>shared/lib/http-client.ts</code>	Fetch con auth automática	HTTP Client / Middleware
<code>app/app.tsx</code>	Rutas y providers	routes.php / bootstrap
<code>app/protected-route.tsx</code>	Guarda de auth	Middleware de auth
<code>app/app-layout.tsx</code>	Layout compartido	Layout base / master template
<code>sw.ts</code>	Offline + notificaciones push	(sin equivalente en MVC clásico)

Tip final: cuando necesites agregar una nueva feature (por ejemplo, "comentarios en tareas"), el proceso es:

1. `types.ts` : define la interfaz `Comment`
2. `api.ts` : añade `listComments()` , `createComment()` ...
3. `use-comments.ts` : crea el hook con estado + mutaciones
4. **Componente:** crea el componente de UI que use el hook
5. **Ruta** (si necesitas página propia): añádela en `app.tsx`

Cada capa toca lo suyo. La UI no sabe cómo funciona fetch. Los hooks no saben nada de JSX. La API no sabe nada del estado.